

In case of max pooling, we define a spatial neighbourhood and take the largest element from the rectified feature map within that window. In average pooling, we take the average of all elements in that window.

3. Recurrent layer: This is mostly used in sequential and time series modelling.
4. Embedding layers: These are mostly used in Natural Language Processing.
5. Normalisation layers

Keras models are of two types:

1. The sequential model that has been used here.
2. A model with the Functional API.

Keras optimiser is a module that contains different types of backpropagation algorithms for training our model. Some of the famous optimisers are:

1. sgd - stochastic gradient descent optimiser.
2. Rmsprop - root means square propagation optimiser.
3. Adams optimiser.
4. Adagrad optimiser.
5. Adadelta optimiser.

Since we are working with a small dataset but deep learning models require a large dataset. Hence, we are going to augment our data using the image data generator function.

## Model of our Architecture:

We are going to use the architecture of the famous VGG model. The VGG network architecture was introduced by Zisserman and Simonyan in 2014. Their paper was - Very Deep Convolutional Networks for Large Scale Image Recognition.

This network is known for its simplicity, using only  $3 \times 3$  convolutional layers stacked on top of each other in increasing depth. Max pooling layer is used to reduce volume size. A softmax classifier that is finally used to classify the images then follows two fully connected layers, each with 4,096 nodes. The filter size increases as we go down layers.

Each convolutional layer is followed by a max pooling layer. After pooling comes flattening. It converts the matrix into a linear array to input it into the nodes of our neural network. We would be creating a sequential model. This tells keras to stack all layers sequentially – one on top of another. The sequence of steps to be followed to develop this model are:

1. We create the first layer by calling the .add() function on the model that